



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4
по дисциплине «Системы анализа больших данных»

Студент группы *ИНБО-21-23. Илюхин К.Н.*

(подпись)

Преподаватель *Алексеева Е.С.*

(подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
1 ВВЕДЕНИЕ.....	4
1.1 Цель и постановка задачи.....	4
2 ХОД РАБОТЫ	5
ВЫВОДЫ	17

1 ВВЕДЕНИЕ

1.1 Цель и постановка задачи

Целью практической работы является формирование навыков реализации приложений для распределённых вычислений с применением библиотеки PySpark на основе кластера Arenadata Hyperwave.

В рамках работы были выполнены следующие задачи:

- подготовка исходного датасета Brazilian E-Commerce (Olist) и загрузка его в HDFS;
- разработка PySpark-приложения для чтения, очистки и преобразования данных;
- формирование аналитической модели данных (таблица фактов и таблицы измерений);
- расчёт аналитических витрин;
- сохранение результатов в HDFS;
- проверка выполнения приложения через Spark History Server.

2 ХОД РАБОТЫ

Исходный датасет Olist был загружен в PostgreSQL в практической работе №2. Для использования в PySpark необходимо экспортировать таблицы в CSV-формат. Экспорт выполнялся с помощью команды `\copy` от имени пользователя postgres на ВМ ubuntu-cass (IP 192.168.0.118 / 192.168.58.110).

Были экспортированы 8 таблиц: orders, order_items, customers, products, sellers, order_payments, order_reviews, product_category_name_translation.

CSV-файлы были переданы на узел adh-m1 по scp и загружены в HDFS в директорию `/user/user/olist/raw`. Права на директорию были выданы пользователю user через `sudo -u hdfs`.

```
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/raw
Found 8 items
-rw-r--r--  3 user user      8562270 2026-09-30 18:39 /user/user/olist/raw/olist_customers.csv
-rw-r--r--  3 user user    15007593 2026-09-30 18:39 /user/user/olist/raw/olist_order_items.csv
-rw-r--r--  3 user user     5647780 2026-09-30 18:39 /user/user/olist/raw/olist_order_payments.csv
-rw-r--r--  3 user user    33582526 2026-09-30 18:39 /user/user/olist/raw/olist_order_reviews.csv
-rw-r--r--  3 user user    17406884 2026-09-30 18:39 /user/user/olist/raw/olist_orders.csv
-rw-r--r--  3 user user       2540 2026-09-30 18:39 /user/user/olist/raw/olist_product_category.csv
-rw-r--r--  3 user user    2337818 2026-09-30 18:39 /user/user/olist/raw/olist_products.csv
-rw-r--r--  3 user user     163589 2026-09-30 18:39 /user/user/olist/raw/olist_sellers.csv
```

Рисунок 1 — список 8 CSV в HDFS

```
user@adh-m1:~$ hdfs dfs -cat /user/user/olist/raw/olist_orders.csv | head -1
order_id,customer_id,order_status,order_purchase_timestamp,order_approved_at,order_delivered_carrier_date,order_delivered_customer_date,order_estimated_delivery_date
cat: Unable to write to output stream.
user@adh-m1:~$ hdfs dfs -cat /user/user/olist/raw/olist_order_items.csv | head -1
order_id,order_item_id,product_id,seller_id,shipping_limit_date,price,freight_value
cat: Unable to write to output stream.
user@adh-m1:~$ hdfs dfs -cat /user/user/olist/raw/olist_customers.csv | head -1
customer_id,customer_unique_id,customer_zip_code_prefix,customer_city,customer_state
cat: Unable to write to output stream.
user@adh-m1:~$ hdfs dfs -cat /user/user/olist/raw/olist_products.csv | head -1
product_id,product_category_name,product_name_lenght,product_description_lenght,product_photos_qty,product_weight_g,product_length_cm,product_height_cm,product_width_cm
cat: Unable to write to output stream.
user@adh-m1:~$ hdfs dfs -cat /user/user/olist/raw/olist_sellers.csv | head -1
seller_id,seller_zip_code_prefix,seller_city,seller_state
cat: Unable to write to output stream.
user@adh-m1:~$ hdfs dfs -cat /user/user/olist/raw/olist_product_category.csv | head -1
product_category_name,product_category_name_english
user@adh-m1:~$
```

Рисунок 2 — заголовки CSV (структура данных)

Перед запуском приложения была проверена версия Spark и доступность команды `spark3-submit`.

```

user@adh-m1:~$ export JAVA_HOME=/usr/lib/jvm/java-arenadata-openjdk-8
user@adh-m1:~$ export HADOOP_CONF_DIR=/etc/hadoop/conf
user@adh-m1:~$ spark3-submit --version
Welcome to

  ____      _
 / ___|  _ \| | | |
 \___ \| |_) | |_| |
  ___) | |_) | | | |
 |____|_|_|\___|_|_|_|

version 3.5.4

Using Scala version 2.13.8, OpenJDK 64-Bit Server VM, 1.8.0_412
Branch adh
Compiled by user on 2026-03-19T14:12:11Z
Revision 4e44d2910cc43b4bfc58ecb571b75bccd95b6bf6
Url git@ssh.gitlab.adsw.io:arenadata/infrastructure/code/ci/prj_adh.git
Type --help for more information.

```

Рисунок 3 — вывод `spark3-submit --version` (Spark 3.5.4, Java 11)

Приложение `lab4_olist.py` реализует следующую логику:

1. Создание `SparkSession` с именем `lab4_olist`.
2. Чтение 6 CSV из HDFS: `orders`, `order_items`, `customers`, `products`, `sellers`, `product_category`.
3. Staging-слой: JOIN всех таблиц в один `DataFrame` `stage_olist`.
4. Очистка:
 - Фильтр `order_status = 'delivered'`.
 - `price > 0`.
 - `order_purchase_timestamp` не NULL.
 - Вычисление `line_amount = price + freight_value`.
 - Парсинг `order_purchase_ts` (timestamp) и `order_date` (date).
5. Формирование измерений:
 - `dim_customer` — уникальные клиенты.
 - `dim_product` — уникальные товары.
 - `dim_seller` — уникальные продавцы.
6. Таблица фактов: `fact_sales` — позиции заказов с суммой `line_amount`.
7. Аналитические витрины через `spark.sql`:
 - `mart_monthly_revenue` — выручка по месяцам.
 - `mart_top_products` — топ-10 категорий товаров по выручке.
 - `mart_revenue_by_state` — выручка по штатам клиентов.

8. Сохранение в HDFS:

- fact_sales, dim_* — Parquet.
- mart_* — CSV (coalesce(1), header=True).

9. Вывод в консоль — для контроля результатов.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, trim, to_timestamp, to_date, date_trunc,
    round as _round, count, countDistinct, sum as _sum,
    monotonically_increasing_id, first
)

# 1. Создание SparkSession
spark = SparkSession.builder \
    .appName("lab4_olist") \
    .getOrCreate()

# 2. Пути
input_base = "hdfs:///user/user/olist/raw"
out_base = "hdfs:///user/user/olist/out"

# 3. Чтение CSV из HDFS
orders = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_orders.csv")

order_items = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_order_items.csv")

customers = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_customers.csv")
```

Рисунок 4 — код lab4_olist.py часть 1

```
GNU nano 6.2 /home/user/lab4_olist.py
order_items = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_order_items.csv")

customers = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_customers.csv")

products = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_products.csv")

sellers = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_sellers.csv")

product_category = spark.read.option("header", True).option("inferSchema", True) \
    .csv(f"{input_base}/olist_product_category.csv")

print("=== RAW DATA ===")
print("Orders:", orders.count())
print("Order items:", order_items.count())
print("Customers:", customers.count())
print("Products:", products.count())
print("Sellers:", sellers.count())
print("Product categories:", product_category.count())

orders.show(5, truncate=False)
```

Рисунок 5 — код lab4_olist.py часть 2

```

GNU nano 0.2 /home/user/lab4_olist.py
# 4. Staging-слой: JOIN всех таблиц
stage_olist = orders \
    .join(order_items, "order_id", "inner") \
    .join(customers, "customer_id", "inner") \
    .join(products, "product_id", "inner") \
    .join(sellers, "seller_id", "inner") \
    .join(product_category, "product_category_name", "left")

print("=== STAGE DATA ===")
print("Rows:", stage_olist.count())
stage_olist.printSchema()
stage_olist.show(5, truncate=False)

# 5. Очистка и подготовка данных
clean_df = stage_olist \
    .filter(col("order_status") == "delivered") \
    .filter(col("price") > 0) \
    .filter(col("order_purchase_timestamp").isNotNull()) \
    .withColumn("order_purchase_ts",
                to_timestamp(col("order_purchase_timestamp"),
                             "yyyy-MM-dd HH:mm:ss")) \
    .withColumn("order_date", to_date(col("order_purchase_ts"))) \
    .withColumn("line_amount",
                _round(col("price") + col("freight_value"), 2))

print("=== CLEAN DATA ===")

```

Рисунок 6 — код lab4_olist.py часть 3

```

print("=== CLEAN DATA ===")
print("Rows:", clean_df.count())
clean_df.show(5, truncate=False)

# 6. Измерения
dim_customer = clean_df.select(
    "customer_id", "customer_city", "customer_state"
).dropDuplicates()

dim_product = clean_df.select(
    "product_id", "product_category_name", "product_category_name_english"
).dropDuplicates()

dim_seller = clean_df.select(
    "seller_id", "seller_city", "seller_state"
).dropDuplicates()

# 7. Таблица фактов
fact_sales = clean_df.select(
    monotonically_increasing_id().alias("sale_id"),
    "order_id",
    "order_item_id",
    "customer_id",
    "product_id",
    "seller_id",
    "order_purchase_ts",

```

Рисунок 7 — код lab4_olist.py часть 4

```

GNU nano 6.2 /home/user/lab4_olist.py
)

print("=== DIM CUSTOMER ===")
print("Rows:", dim_customer.count())
dim_customer.show(5, truncate=False)

print("=== DIM PRODUCT ===")
print("Rows:", dim_product.count())
dim_product.show(5, truncate=False)

print("=== DIM SELLER ===")
print("Rows:", dim_seller.count())
dim_seller.show(5, truncate=False)

print("=== FACT SALES ===")
print("Rows:", fact_sales.count())
fact_sales.show(5, truncate=False)

# 8. Временные представления
fact_sales.createOrReplaceTempView("fact_sales")
dim_customer.createOrReplaceTempView("dim_customer")
dim_product.createOrReplaceTempView("dim_product")
dim_seller.createOrReplaceTempView("dim_seller")

# 9. Аналитические витрины
mart_monthly_revenue = spark.sql("""

```

Рисунок 8 — код lab4_olist.py часть 5

```

mart_monthly_revenue = spark.sql("""
    SELECT
        date_trunc('month', order_purchase_ts) AS month_ts,
        ROUND(SUM(line_amount), 2) AS revenue,
        COUNT(DISTINCT order_id) AS orders_cnt
    FROM fact_sales
    GROUP BY 1
    ORDER BY 1
""")

mart_top_products = spark.sql("""
    SELECT
        p.product_category_name_english AS product_category,
        ROUND(SUM(f.line_amount), 2) AS revenue,
        COUNT(*) AS items_cnt
    FROM fact_sales f
    JOIN dim_product p ON f.product_id = p.product_id
    WHERE p.product_category_name_english IS NOT NULL
    GROUP BY p.product_category_name_english
    ORDER BY revenue DESC
    LIMIT 10
""")

mart_revenue_by_state = spark.sql("""
    SELECT
        c.customer_state,

```

Рисунок 9 — код lab4_olist.py часть 6

```

mart_revenue_by_state = spark.sql("""
    SELECT
        c.customer_state,
        ROUND(SUM(f.line_amount), 2) AS revenue,
        COUNT(DISTINCT f.order_id) AS orders_cnt
    FROM fact_sales f
    JOIN dim_customer c ON f.customer_id = c.customer_id
    GROUP BY c.customer_state
    ORDER BY revenue DESC
""")

print("=== MART MONTHLY REVENUE ===")
mart_monthly_revenue.show(30, truncate=False)

print("=== MART TOP PRODUCTS ===")
mart_top_products.show(10, truncate=False)

print("=== MART REVENUE BY STATE ===")
mart_revenue_by_state.show(30, truncate=False)

# 10. Сохранение результатов в HDFS
fact_sales.write.mode("overwrite").parquet(f"{out_base}/fact_sales")
dim_customer.write.mode("overwrite").parquet(f"{out_base}/dim_customer")
dim_product.write.mode("overwrite").parquet(f"{out_base}/dim_product")
dim_seller.write.mode("overwrite").parquet(f"{out_base}/dim_seller")

```

Рисунок 10 — код lab4_olist.py часть 7

Запуск выполнялся в локальном режиме local[*]:

```

user@adh-m1:~$ spark3-submit --master yarn --deploy-mode client ~/lab4_olist.py
26/09/30 18:54:21 INFO SparkContext: Running Spark version 3.5.4
26/09/30 18:54:21 INFO SparkContext: OS info Linux, 5.15.0-173-generic, amd64
26/09/30 18:54:21 INFO SparkContext: Java version 1.8.0_412
26/09/30 18:54:22 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java
classes where applicable
26/09/30 18:54:22 INFO ResourceUtils: =====
26/09/30 18:54:22 INFO ResourceUtils: No custom resources configured for spark.driver.
26/09/30 18:54:22 INFO ResourceUtils: =====
26/09/30 18:54:22 INFO SparkContext: Submitted application: lab4_olist
26/09/30 18:54:22 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(cores -> name: cores,
amount: 1, script: , vendor: , memory -> name: memory, amount: 1024, script: , vendor: , offHeap -> name: offHeap, amount: 0, script: , vendor: ), task resources: Map(cpus -> name: cpus, amount: 1.0)
26/09/30 18:54:22 INFO ResourceProfile: Limiting resource is cpus at 1 tasks per executor

```

Рисунок 11 — вывод spark3-submit + PySpark job completed successfully

В результате выполнения приложения были получены следующие данные:

Исходные данные:

- Заказы (orders): 99 441
- Позиции (order_items): 112 650
- Клиенты (customers): 99 441
- Товары (products): 32 951
- Продавцы (sellers): 3 095
- Fact sales (после очистки): 110 197

Витрина mart_monthly_revenue (23 месяца):

- 2017-11: 1 153 364.20 (пик)
- 2018-08: 985 491.64

Витрина mart_top_products (топ-10 категорий):

- health_beauty: 1 412 089.53 (9 465)
- watches_gifts: 1 264 333.12 (5 859)
- bed_bath_table: 1 225 209.26 (10 953)
- sports_leisure: 1 118 256.91 (8 431)
- computers_accessories: 1 032 723.77 (7 644)

Витрина mart_revenue_by_state (топ-5 штатов):

- SP: 5 769 703.15 (40 501 заказ)
- RJ: 2 055 401.57 (12 350)
- MG: 1 818 891.67 (11 354)
- RS: 861 472.79 (5 345)
- PR: 781 708.80 (4 923)

```
user@adh-m1: ~  
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out  
Found 7 items  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/dim_customer  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/dim_product  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/dim_seller  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/fact_sales  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/mart_monthly_revenue  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/mart_revenue_by_state  
drwxr-xr-x - user user      0 2026-09-30 19:59 /user/user/olist/out/mart_top_products  
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/fact_sales  
Found 3 items  
-rw-r--r--  3 user user      0 2026-09-30 19:59 /user/user/olist/out/fact_sales/_SUCCESS  
-rw-r--r--  3 user user  6735669 2026-09-30 19:59 /user/user/olist/out/fact_sales/part-00000-3e2d36c6-7aef-4e5c-ac8b-  
371f2b1ecc04-c000.snappy.parquet  
-rw-r--r--  3 user user  3982982 2026-09-30 19:59 /user/user/olist/out/fact_sales/part-00001-3e2d36c6-7aef-4e5c-ac8b-  
371f2b1ecc04-c000.snappy.parquet  
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/dim_customer  
Found 3 items  
-rw-r--r--  3 user user      0 2026-09-30 19:59 /user/user/olist/out/dim_customer/_SUCCESS  
-rw-r--r--  3 user user  1690723 2026-09-30 19:59 /user/user/olist/out/dim_customer/part-00000-db2e373c-55d4-4a37-a7e  
9-84c8c0735f9c-c000.snappy.parquet  
-rw-r--r--  3 user user  1724495 2026-09-30 19:59 /user/user/olist/out/dim_customer/part-00001-db2e373c-55d4-4a37-a7e  
9-84c8c0735f9c-c000.snappy.parquet  
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/dim_product  
Found 3 items  
-rw-r--r--  3 user user      0 2026-09-30 19:59 /user/user/olist/out/dim_product/_SUCCESS  
-rw-r--r--  3 user user   550646 2026-09-30 19:59 /user/user/olist/out/dim_product/part-00000-7043a300-6faa-437e-88fb-  
760c0a6bf24f-c000.snappy.parquet  
-rw-r--r--  3 user user   562365 2026-09-30 19:59 /user/user/olist/out/dim_product/part-00001-7043a300-6faa-437e-88fb-  
760c0a6bf24f-c000.snappy.parquet
```

Рисунок 12 — результаты в HDFS (hdfs dfs -ls, hdfs dfs -cat)

```

-rw-r--r-- 3 user user 1724495 2026-09-30 19:59 /user/user/olist/out/dim_customer/part-00001-db2e373c-55d4-4a37-a7e
9-84c8c0735f9c-c000.snappy.parquet
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/dim_product
Found 3 items
-rw-r--r-- 3 user user 0 2026-09-30 19:59 /user/user/olist/out/dim_product/_SUCCESS
-rw-r--r-- 3 user user 550646 2026-09-30 19:59 /user/user/olist/out/dim_product/part-00000-7043a300-6faa-437e-88fb
-760c0a6bf24f-c000.snappy.parquet
-rw-r--r-- 3 user user 562365 2026-09-30 19:59 /user/user/olist/out/dim_product/part-00001-7043a300-6faa-437e-88fb
-760c0a6bf24f-c000.snappy.parquet
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/dim_seller
Found 2 items
-rw-r--r-- 3 user user 0 2026-09-30 19:59 /user/user/olist/out/dim_seller/_SUCCESS
-rw-r--r-- 3 user user 109806 2026-09-30 19:59 /user/user/olist/out/dim_seller/part-00000-4652ef13-894d-414f-a6fc-
4a62c2e1bf9d-c000.snappy.parquet
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/mart_monthly_revenue
Found 2 items
-rw-r--r-- 3 user user 0 2026-09-30 19:59 /user/user/olist/out/mart_monthly_revenue/_SUCCESS
-rw-r--r-- 3 user user 936 2026-09-30 19:59 /user/user/olist/out/mart_monthly_revenue/part-00000-df268162-6eda-
4e09-9115-39d366b0f6ec-c000.csv
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/mart_top_products
Found 2 items
-rw-r--r-- 3 user user 0 2026-09-30 19:59 /user/user/olist/out/mart_top_products/_SUCCESS
-rw-r--r-- 3 user user 327 2026-09-30 19:59 /user/user/olist/out/mart_top_products/part-00000-89a86e76-b3b3-46b
b-801f-4011e8f8f9d4-c000.csv
user@adh-m1:~$ hdfs dfs -ls /user/user/olist/out/mart_revenue_by_state
Found 2 items
-rw-r--r-- 3 user user 0 2026-09-30 19:59 /user/user/olist/out/mart_revenue_by_state/_SUCCESS
-rw-r--r-- 3 user user 497 2026-09-30 19:59 /user/user/olist/out/mart_revenue_by_state/part-00000-86d6dcdd-9bd1
-4546-9bc8-69291610a139-c000.csv
user@adh-m1:~$

```

Рисунок 13 — результаты в HDFS (hdfs dfs -ls, hdfs dfs -cat)

```

user@adh-m1:~$ hdfs dfs -cat /user/user/olist/out/mart_monthly_revenue/part-*.csv | head -30
month_ts,revenue,orders_cnt
2016-09-01T00:00:00.000Z,143.46,1
2016-10-01T00:00:00.000Z,46490.66,265
2016-12-01T00:00:00.000Z,19.62,1
2017-01-01T00:00:00.000Z,127482.37,750
2017-02-01T00:00:00.000Z,271239.32,1653
2017-03-01T00:00:00.000Z,414330.95,2546
2017-04-01T00:00:00.000Z,390812.4,2303
2017-05-01T00:00:00.000Z,566851.4,3546
2017-06-01T00:00:00.000Z,490050.37,3135
2017-07-01T00:00:00.000Z,566299.08,3872
2017-08-01T00:00:00.000Z,645832.36,4193
2017-09-01T00:00:00.000Z,701077.49,4150
2017-10-01T00:00:00.000Z,751117.01,4478
2017-11-01T00:00:00.000Z,1153364.2,7289
2017-12-01T00:00:00.000Z,843078.29,5513
2018-01-01T00:00:00.000Z,1077887.46,7069
2018-02-01T00:00:00.000Z,966168.41,6555
2018-03-01T00:00:00.000Z,1120598.24,7003
2018-04-01T00:00:00.000Z,1132878.93,6798
2018-05-01T00:00:00.000Z,1128774.52,6749
2018-06-01T00:00:00.000Z,1011978.29,6099
2018-07-01T00:00:00.000Z,1027807.28,6159
2018-08-01T00:00:00.000Z,985491.64,6351

```

Рисунок 14 — результаты в HDFS (hdfs dfs -ls, hdfs dfs -cat)

```

user@adh-m1:~$ hdfs dfs -cat /user/user/olist/out/mart_top_products/part-*.csv
product_category,revenue,items_cnt
health_beauty,1412089.53,9465
watches_gifts,1264333.12,5859
bed_bath_table,1225209.26,10953
sports_leisure,1118256.91,8431
computers_accessories,1032723.77,7644
furniture_decor,880329.92,8160
housewares,758392.25,6795
cool_stuff,691680.89,3718
auto,669454.75,4140
garden_tools,567145.68,4268

```

Рисунок 15 — результаты в HDFS (hdfs dfs -ls, hdfs dfs -cat)

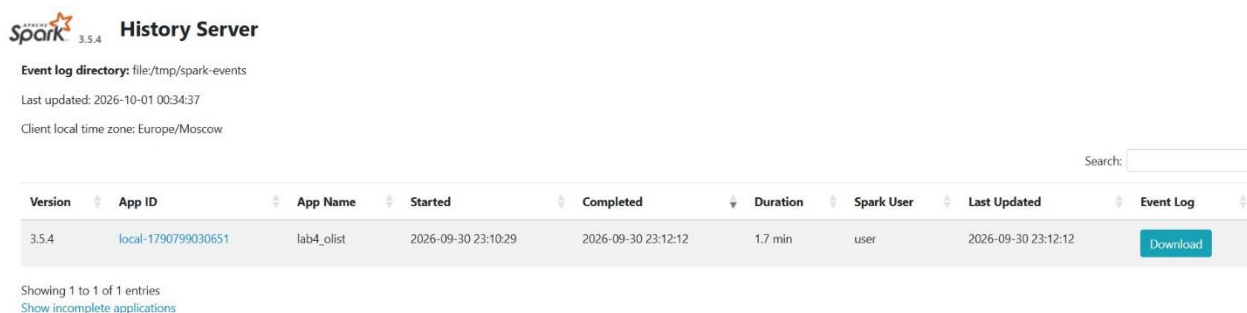
```

user@adh-m1:~$ hdfs dfs -cat /user/user/olist/out/mart_revenue_by_state/part-*.csv | head -30
customer_state,revenue,orders_cnt
SP,5769703.15,40501
RJ,2055401.57,12350
MG,1818891.67,11354
RS,861472.79,5345
PR,781708.8,4923
SC,595127.78,3546
BA,591137.81,3256
DF,346123.35,2080
GO,334212.35,1957
ES,317657.93,1995
PE,308972.05,1593
CE,266436.77,1279
PA,212023.57,946
MT,181224.42,886
MA,147803.55,717
PB,137838.55,517
MS,134367.55,701
PI,105178.19,476
RN,100714.78,474
AL,94172.49,397
SE,70289.13,335
TO,60007.37,274
RO,56966.0,243
AM,27585.47,145
AC,19575.33,80
AP,16141.81,67
RR,9039.52,41

```

Рисунок 16 — результаты в HDFS (hdfs dfs -ls, hdfs dfs -cat)

После завершения приложения Spark записал event log в директорию /tmp/spark-events. Spark History Server, настроенный на чтение этой директории, отобразил информацию о выполненном приложении.



The screenshot shows the Spark History Server interface. At the top, it says "History Server" and "Event log directory: file:/tmp/spark-events". Below this, it shows "Last updated: 2026-10-01 00:34:37" and "Client local time zone: Europe/Moscow". There is a search bar on the right. Below the search bar is a table with the following columns: Version, App ID, App Name, Started, Completed, Duration, Spark User, Last Updated, and Event Log. The table contains one entry for version 3.5.4, App ID local-1790799030651, App Name lab4_olist, Started 2026-09-30 23:10:29, Completed 2026-09-30 23:12:12, Duration 1.7 min, Spark User user, and Last Updated 2026-09-30 23:12:12. There is a "Download" button next to the Event Log column. At the bottom, it says "Showing 1 to 1 of 1 entries" and "Show incomplete applications".

Version	App ID	App Name	Started	Completed	Duration	Spark User	Last Updated	Event Log
3.5.4	local-1790799030651	lab4_olist	2026-09-30 23:10:29	2026-09-30 23:12:12	1.7 min	user	2026-09-30 23:12:12	Download

Рисунок 17 — Список выполненных приложений в Spark History Server

На вкладке Jobs отображается полный список заданий, выполненных в рамках приложения. Всего было выполнено 201 задание, включая чтение CSV-файлов, JOIN таблиц Olist, агрегацию данных и запись результатов в HDFS.

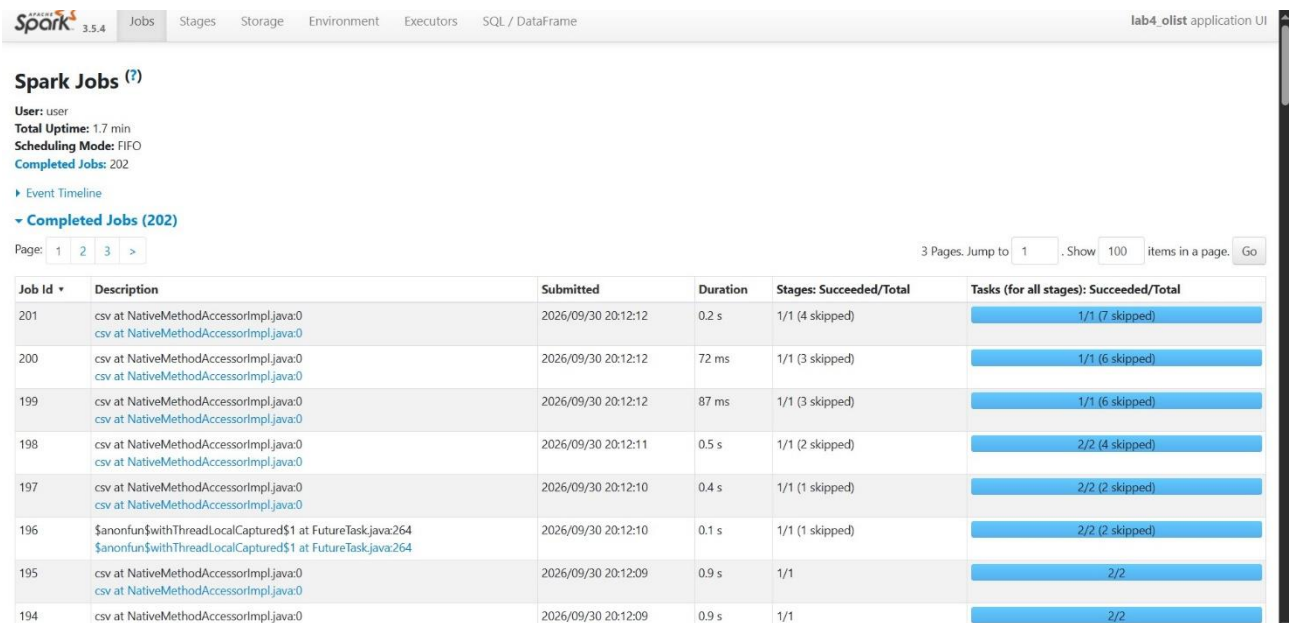


Рисунок 18 — Список заданий (Jobs) приложения lab4_olist в Spark History Server

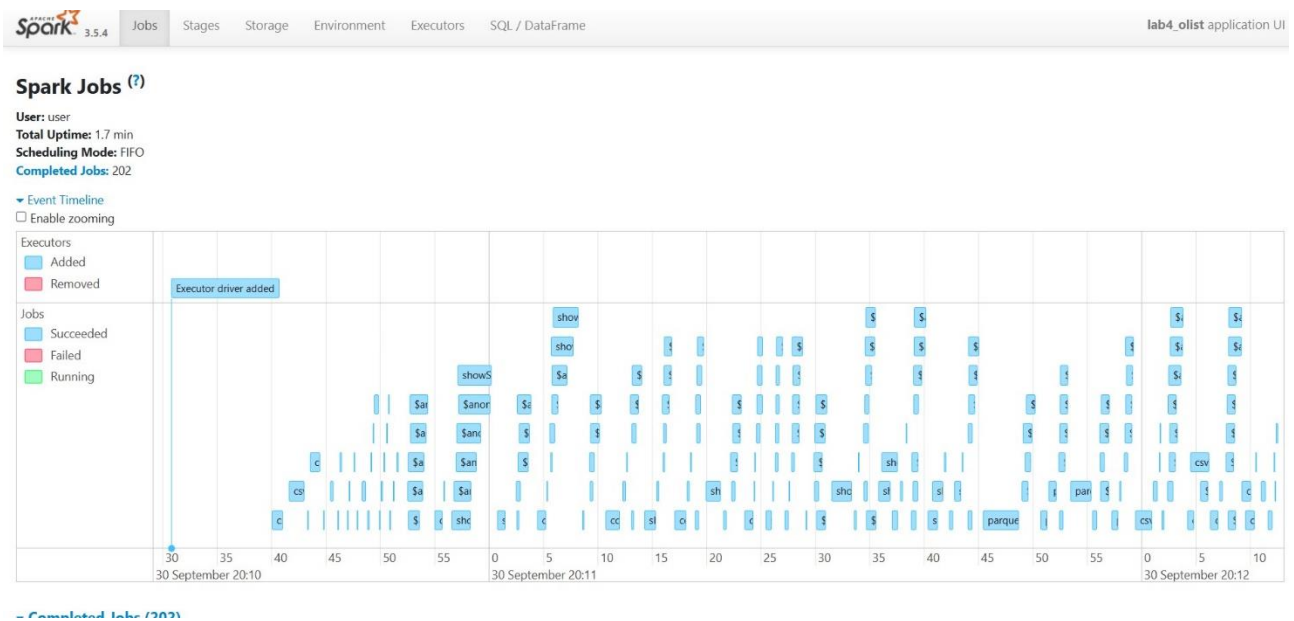


Рисунок 19 — Список заданий (Jobs) приложения lab4_olist в Spark History Server

На вкладке SQL отображаются запросы, выполненные через `spark.sql()`. В приложении было выполнено три аналитических SQL-запроса для расчёта витрин: выручка по месяцам, топ-10 категорий товаров, выручка по штатам.

Spark 3.5.4 Jobs Stages Storage Environment Executors SQL / DataFrame lab4_olist application UI				
SQL / DataFrame				
Completed Queries: 39				
Completed Queries (39)				
Page: 1 1 Pages. Jump to 1 . Show 100 items in a page. Go				
ID	Description	Submitted	Duration	Job IDs
38	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:07	5 s	[187][188][189][190][191][192][193][194][195][196][197][198][199][200][201]
37	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:02	5 s	[175][176][177][178][179][180][181][182][183][184][185][186]
36	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:58	4 s	[165][166][167][168][169][170][171][172][173][174]
35	parquet at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:56	2 s	[159][160][161][162][163][164]
34	parquet at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:52	4 s	[152][153][154][155][156][157][158]
33	parquet at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:48	3 s	[146][147][148][149][150][151]
32	parquet at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:43	5 s	[140][141][142][143][144][145]
31	showString at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:38	5 s	[127][128][129][130][131][132][133][134][135][136][137][138][139]
30	showString at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:34	4 s	[115][116][117][118][119][120][121][122][123][124][125][126]
29	showString at NativeMethodAccessorImpl.java:0	2026/09/30 20:11:30	4 s	[107][108][109][110][111][112][113][114]

Рисунок 20 — SQL-запросы приложения lab4_olist в Spark History Server

На вкладке Stages отображаются этапы выполнения приложения. Каждая стадия включает набор задач (Tasks), выполняемых параллельно. Информация о shuffle-операциях позволяет оценить объём перераспределения данных между партициями.

Spark 3.5.4 Jobs Stages Storage Environment Executors SQL / DataFrame lab4_olist application UI								
Stages for All Jobs								
Completed Stages: 202								
Skipped Stages: 68								
Completed Stages (202)								
Page: 1 2 3 > 3 Pages. Jump to 1 . Show 100 items in a page. Go								
Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
269	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:12	0.2 s	1/1		497.0 B	2.1 KiB	
264	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:12	46 ms	1/1			4.1 KiB	2.1 KiB
260	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:12	68 ms	1/1			4.1 KiB	
256	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:11	0.4 s	2/2			3.9 MiB	4.1 KiB
253	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:10	0.4 s	2/2			6.7 MiB	3.9 MiB
251	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:10	0.1 s	2/2			4.3 MiB	
249	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:09	0.6 s	2/2	14.4 MiB			4.3 MiB
248	csv at NativeMethodAccessorImpl.java:0	2026/09/30 20:12:09	0.9 s	2/2	14.4 MiB			6.7 MiB
247	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	0.3 s	1/1	2.2 MiB			
246	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	0.6 s	2/2	8.2 MiB			
245	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	52 ms	1/1	2.5 KiB			
244	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	0.1 s	1/1	159.8 KiB			
243	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	0.2 s	1/1	2.2 MiB			
242	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	0.4 s	2/2	8.2 MiB			
241	\$anonfun\$withThreadLocalCaptured\$1 at FutureTask.java:264	2026/09/30 20:12:07	0.7 s	2/2	16.7 MiB			

Рисунок 21 — Этапы выполнения (Stages) приложения lab4_olist в Spark History Server

Дополнительно результаты были проверены через веб-интерфейс HDFS NameNode. В разделе Utilities → Browse the file system по пути /user/user/olist/out отображаются 7 директорий: четыре с Parquet-данными (fact_sales, dim_customer,

dim_product, dim_seller) и три с CSV-витринами (mart_monthly_revenue, mart_revenue_by_state, mart_top_products).

Browse Directory

/user/user/olist/out

Go!

Show

25

entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:11	0	0 B	dim_customer	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:11	0	0 B	dim_product	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:11	0	0 B	dim_seller	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:11	0	0 B	fact_sales	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:12	0	0 B	mart_monthly_revenue	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:12	0	0 B	mart_revenue_by_state	
☐	drwxr-xr-x	user	user	0 B	Sep 30 23:12	0	0 B	mart_top_products	

Showing 1 to 7 of 7 entries

Previous

1

Next

Hadoop, 2023.

Рисунок 22 — Содержимое директории /user/user/olist/out в HDFS NameNode UI

ВЫВОДЫ

В ходе практической работы были сформированы навыки реализации приложений для распределённых вычислений с применением библиотеки PySpark на кластере Arenadata Hyperwave.

Были выполнены:

- подготовка датасета Brazilian E-Commerce (Olist) и загрузка его в HDFS;
- разработка PySpark-приложения с использованием DataFrame API и Spark SQL;
- формирование аналитической модели данных (таблица фактов fact_sales, измерения dim_customer, dim_product, dim_seller);
- расчёт трёх аналитических витрин (mart_monthly_revenue, mart_top_products, mart_revenue_by_state);
- сохранение результатов в HDFS в форматах Parquet и CSV;
- проверка выполнения приложения через Spark History Server.